

GRANT, REVOKE, DENY and the hierarchy

Securables, inheritance, the traversal privileges — and an honest look at where DENY actually exists.

In Unity Catalog, **nobody has access to anything by default**. Every permission is something somebody granted, deliberately.

The securable hierarchy

LEVEL	CONTAINS
Metastore	The top-level container. Catalogs, and the objects below.
Catalog	Schemas. The first level of the three-part name.
Schema	Tables, views, volumes, functions, models.
Table / view / volume / function	The objects people actually query

ON THE EXAM

Also securable, and outside the three-level namespace: external locations, storage credentials, connections and shares.

Inheritance

- ◆ **Privileges are inherited downward.** A grant on a catalog applies to every schema and object in it.
- ◆ **Including future objects.** A table created tomorrow inherits today's catalog grant.
- ◆ **The highest level inherited from is the catalog.**
- ◆ **Privileges granted on a metastore are not inherited.**

ON THE EXAM

Granting at catalog level is powerful and blunt. It covers everything, including tables nobody has created yet.

SELECT on a table is not enough. You also need **USE CATALOG** on its catalog and **USE SCHEMA** on its schema, or you cannot reach it.

Privileges worth knowing

PRIVILEGE	ALLOWS
USE CATALOG	Traverse into a catalog. Required to reach anything inside it.
USE SCHEMA	Traverse into a schema. Required to reach anything inside it.
SELECT	Read a table or view
MODIFY	Insert, update and delete data in a table
CREATE TABLE	Create a table in a schema
EXECUTE	Run a function — including a registered model
READ VOLUME / WRITE VOLUME	Read from or write to a volume
BROWSE	See an object's metadata without access to its data

ON THE EXAM

BROWSE is how a data catalog stays discoverable. People can find that a table exists and request access, without reading it.

ALL PRIVILEGES

- ◆ **Implies all applicable privileges** for that object type, without granting each individually.
- ◆ **It deliberately excludes some.** `MANAGE`, `EXTERNAL USE SCHEMA` and `EXTERNAL USE LOCATION` are not included.
- ◆ **The reason is to prevent accidental data exfiltration or privilege escalation.**
- ◆ **So ALL PRIVILEGES is not quite all privileges**, which is exactly the kind of thing an exam asks about.

ON THE EXAM

Schema owners do not get `EXTERNAL USE SCHEMA` by default either, and external location owners do not get `EXTERNAL USE LOCATION`.

GRANT and REVOKE

- ◆ GRANT privilege ON securable-type name TO principal
- ◆ REVOKE privilege ON securable-type name FROM principal
- ◆ Both are available in SQL and in Catalog Explorer's Permissions tab.
- ◆ **REVOKE succeeds even if the privilege was never granted.** It ensures absence rather than reversing a grant.

ON THE EXAM

On a metastore grant you omit the metastore name, because the metastore attached to your workspace is assumed.

The exam guide names DENY. Databricks documentation says: **no support for DENY** in Unity Catalog. Both of those are true, and you need both.

Where DENY does exist

- ◆ **In the legacy Hive metastore table access control model**, alongside GRANT, REVOKE, MSCK and SHOW GRANTS.
- ◆ **There, denying a privilege on a catalog denies it on every schema within**, the same way grants inherit.
- ◆ **A denied user cannot even see the object** when listing.
- ◆ **That model is legacy**, and Databricks recommends upgrading to Unity Catalog.

ON THE EXAM

DENY belongs to hive_metastore. If a scenario is set in Unity Catalog, the answer involves not granting, rather than denying.

How to handle this on the day

- ◆ **Know what DENY does**, because the guide names it and a question may use it.
- ◆ **Know that Unity Catalog does not support it**, because that may itself be the answer.
- ◆ **Do not assume a DENY option is automatically wrong**. Read whether the scenario is UC or hive_metastore.
- ◆ **In a UC scenario, achieving exclusion means revoking or not granting**, or narrowing the grant to a smaller scope.

ON THE EXAM

This is the clearest case in the course of the exam guide and the product documentation disagreeing. Carry both.

Every securable has exactly one owner.
Owners can do everything on their object
— but Databricks **does not explicitly grant them ALL PRIVILEGES.**

MANAGE is not ownership

- ◆ **MANAGE grants the ability to grant and revoke privileges** on an object.
- ◆ **It does not grant all privileges on the object.** A user with MANAGE cannot necessarily read the data.
- ◆ **But they can grant themselves SELECT**, which makes the distinction largely theoretical in practice.
- ◆ **Owners can give MANAGE to others**, delegating permission administration without transferring ownership.

ON THE EXAM

MANAGE is how you let a team lead administer access to their own data without handing over the object itself.

Who can grant

- ◆ The owner of the object.
- ◆ The owner of the catalog or schema that contains it.
- ◆ A principal with **MANAGE** on the object.
- ◆ A **metastore admin**, and account admins have default privileges for managing Unity Catalog.

ON THE EXAM

Note the second line. Owning a schema lets you grant on tables inside it, even ones you did not create.

Principals

- ◆ **Users, service principals and groups** — the three the objective names.
- ◆ **Grant to groups, not to individuals.** Joining and leaving a team then changes access automatically.
- ◆ **Unity Catalog uses account-level groups**, whereas the legacy Hive metastore used workspace-local groups.
- ◆ **Service principals are principals too**, which is how the production jobs from module 19 get their access.

ON THE EXAM

Granting to individuals produces an access model nobody can audit, and one that breaks every time somebody changes team.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
SELECT was granted but the query fails	Missing USE CATALOG or USE SCHEMA
A grant should cover future tables too	Grant at catalog or schema level; inheritance is downward
A metastore grant did not reach a catalog	Metastore grants are not inherited
Exclude a group in Unity Catalog	Revoke or do not grant. UC has no DENY.
The scenario is in hive_metastore	DENY exists there, and inherits downward
Someone must administer access but not read data	MANAGE, not ownership
ALL PRIVILEGES did not include something	MANAGE and the EXTERNAL USE privileges are excluded

ON THE EXAM

Seven lines. The first is the most common real-world problem; the fourth and fifth are the pair to keep straight.

An analyst is granted SELECT on a table in finance.reporting. SHOW GRANTS confirms the grant exists and names her group correctly.

Every query she runs against the table fails with a permissions error. Other members of the same group can query it without difficulty.

What is the most likely explanation?

- ☐ A The grant needs time to propagate across the metastore
- ☐ B She is missing USE CATALOG on finance or USE SCHEMA on reporting, which the others hold from another group
- ☐ C A DENY on the table is overriding her SELECT
- ☐ D The table is external, so SELECT does not apply

◆ PAUSE AND DECIDE

What is the most likely explanation?

- A

 Propagation delay
Invents a delay, and would not explain why other members of the same group succeed right now.
- B

 Missing USE CATALOG or USE SCHEMA
Correct. Reaching a table requires traversal privileges on its catalog and schema as well as SELECT. Her colleagues hold those from elsewhere.
- C

 A DENY is overriding it
Unity Catalog has no DENY. Its model is least privilege — unGRANTED is implicitly denied, and there is nothing to override with.
- D

 SELECT does not apply to external tables
SELECT applies to both managed and external tables. The table type does not change how privileges work.

THE SIGNAL

Same group, same grant, different outcome. The difference must come from a privilege held outside that group.

A catalog contains forty schemas. A group has been granted SELECT at catalog level, which they need for thirty-nine of them. One schema holds salary data that this group must not see.

An engineer proposes DENY SELECT on that one schema, so the catalog-level grant stays in place.

What should they do instead?

- ☐ A Use DENY as proposed; it is the standard way to carve out an exception
- ☐ B Unity Catalog has no DENY. Revoke the catalog-level grant and grant at the level of the 39 schemas, or on the objects within them.
- ☐ C Move the salary schema into the hive_metastore catalog so DENY can be used
- ☐ D Grant ALL PRIVILEGES on the catalog and rely on ownership to exclude the schema

◆ PAUSE AND DECIDE

What should they do instead?

A Use DENY

It is the standard way in many database systems, and Unity Catalog does not support it. The proposal would simply fail.

C Move it to hive_metastore

Solves it by moving sensitive data into a legacy model Databricks recommends migrating away from. Precisely the wrong direction for salary data.

B Narrow the grant instead

Correct. With no DENY, exclusion is achieved by granting at a narrower scope — the schemas they should see, rather than the catalog.

D ALL PRIVILEGES plus ownership

ALL PRIVILEGES at catalog level makes the exposure broader, not narrower. Ownership does not exclude anyone from an inherited grant.

THE SIGNAL

Inheritance is downward and there is no mechanism to interrupt it. So the grant must start below the thing you are excluding.

A data platform team owns every catalog. Team leads keep raising tickets asking for colleagues to be granted access to their own team's tables, and the platform team has become a bottleneck.

They want team leads to administer access to their own data, without giving them the ability to drop objects or transfer ownership.

What should they grant?

- ☐ A Transfer ownership of each schema to the relevant team lead
- ☐ B Grant `MANAGE` on the relevant schemas to the team leads
- ☐ C Grant `ALL PRIVILEGES` on the relevant schemas to the team leads
- ☐ D Make each team lead a metastore admin

◆ PAUSE AND DECIDE

What should they grant?

A Transfer ownership

Owners can also drop the object and transfer ownership onward, which the requirement explicitly excludes.

C ALL PRIVILEGES

Deliberately excludes MANAGE, so it would not let them grant to others — the one thing they actually need.

B Grant MANAGE on the schemas

Correct. MANAGE confers exactly the ability to grant and revoke privileges on the object, without ownership's other capabilities.

D Metastore admin

Grants administrative reach over the entire metastore to solve a per-schema problem. Wildly broader than the requirement.

THE SIGNAL

Option C is the trap: ALL PRIVILEGES sounds like more than MANAGE, and specifically excludes it.

An audit finds that access to a sensitive catalog has been granted directly to 63 individual users over three years. Nobody can say who should still have access, and four of the users have left the company.

The team is asked to make the access model auditable.

What is the right change?

- ☐ A Grant to account-level groups instead, and revoke the individual grants
- ☐ B Document the 63 grants in a spreadsheet reviewed quarterly
- ☐ C Move the grants to the metastore level so they are managed in one place
- ☐ D Grant ALL PRIVILEGES to a single service principal and route all queries through it

◆ PAUSE AND DECIDE

What is the right change?

- A

 Grant to groups, revoke the individuals
Correct. Group membership becomes the single place access is decided, and leavers lose access through the same process that removes them from the group.
- B

 A spreadsheet reviewed quarterly
Documents the problem rather than fixing it, and adds a second source of truth that will diverge from the real grants.
- C

 Move grants to the metastore level
Metastore privileges are not inherited downward, so this would not grant catalog access at all.
- D

 One service principal for everyone
Destroys attribution entirely. Every query would appear to come from one identity, which is worse for audit, not better.

THE SIGNAL

Sixty-three individual grants is not sixty-three decisions. It is one decision — 'who is on this team' — recorded sixty-three times.

Nothing is permitted until it is granted.
Traversal is a privilege too, inheritance
only flows down, and in Unity Catalog
exclusion means narrowing the grant.